Unit 3.1
Requirements, Design and Solutioning

-Dr.J.Jeyavel

Types of Requirements

- 1.Functional Requirements: These specify what the system should do, including features and functionalities.
- 2. Non-Functional Requirements: These define how the system performs its functions, covering aspects like performance, usability, reliability, and security.
- 3. User Requirements: These focus on the needs and expectations of end-users interacting with the system.

Analysis of Requirements-UML Use Cases

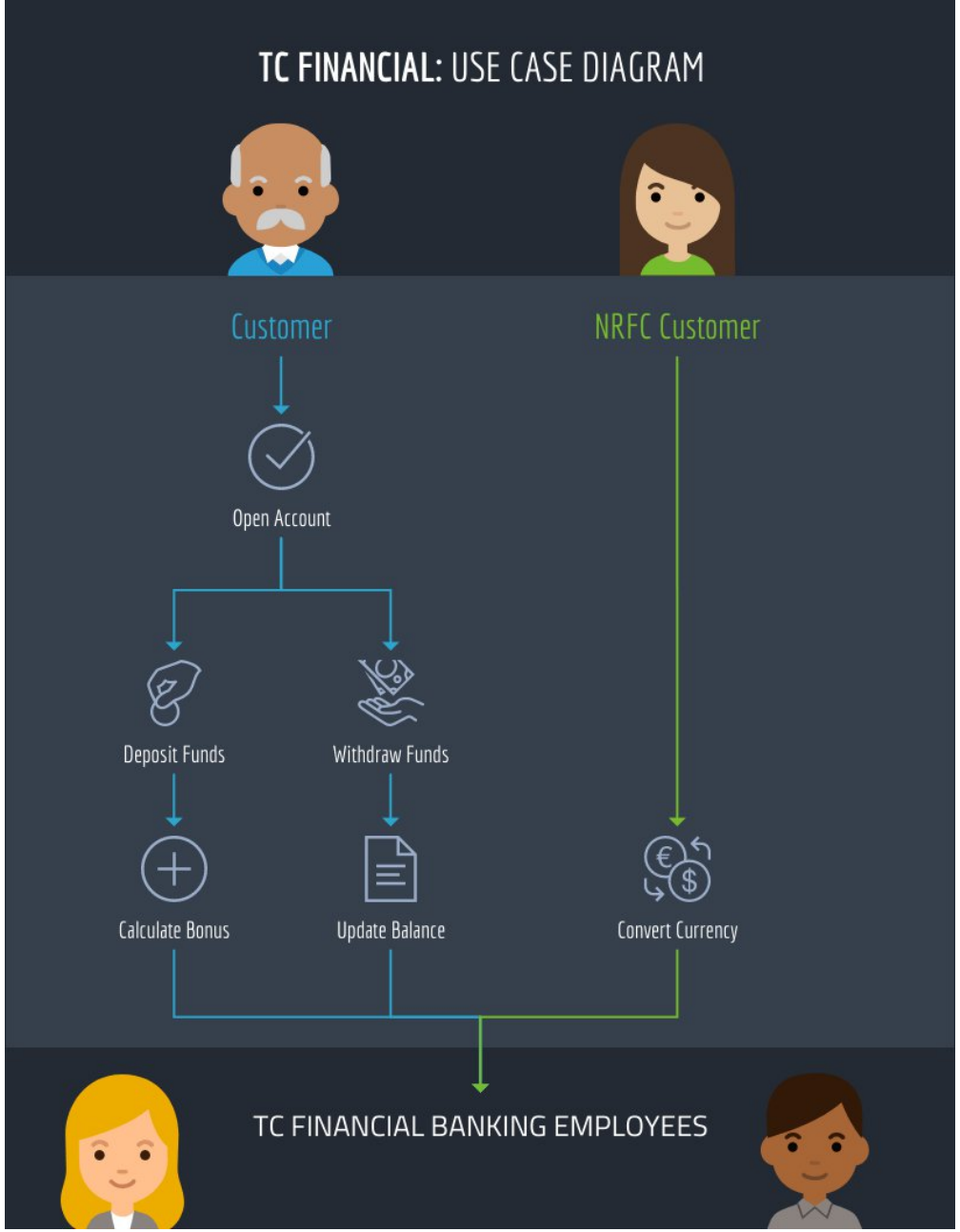
- UML use cases help in visualizing how users (actors) interact with the system. This approach captures functional requirements effectively.
- Components of Use Case Diagrams
- 1. **Actors**: Represent users or external systems that interact with the application (e.g., customers, administrators).
- 2. **Use Cases**: Specific functionalities or actions that the system performs (e.g., login, purchase).
- 3. **System Boundary**: A box that defines the scope of the system, separating use cases from external factors.

Creating Use Case Diagrams

- 1. **Identify Actors**: Determine who will interact with the system.
- 2. **Identify Use Cases**: List all functionalities required by the actors.
- 3. **Connect Actors and Use Cases**: Draw associations between actors and their respective use cases.
- 4. **Define System Boundaries**: Outline what is included within the system’s scope.

Scenarios in Use Case Development

- Example Scenarios
- - **Online Shopping System**:
- - Actors: Customer, Admin.
- - Use Cases: Browse Products, Add to Cart, Checkout.
- - **ATM System**:
- - Actors: Customer, Bank.
- - Use Cases: Withdraw Cash, Check Balance, Deposit Funds.


TC FINANCIAL: USE CASE DIAGRAM
Customer
NRFC Customer
Open Account
Deposit Funds
Withdraw Funds
Calculate Bonus
Update Balance
Convert Currency
TC FINANCIAL BANKING EMPLOYEES

Benefits of Using UML Use Cases

- Clarity: Provides a clear overview of user interactions with the system.
- Communication Tool: Facilitates discussions among stakeholders about system functionality.
- Validation: Helps in validating requirements against user needs during testing phases.

Steps to Identify Actors

- Define the System Scope
- Identify External Entities(People, other systems, organizations)
- Categorize Actors (Primary and Supporting Actors)
- Ask Key Questions(Users, Goals, Interacting systems, User roles)
- Document Each Actor.
- Eliminate Non-Goal Actors

Example of Actor Identification

- Customer- A user who browses products, places orders, and makes payments. Their objective is to complete purchases efficiently and receive order confirmations. |
- Admin- An administrative user responsible for managing user accounts, overseeing transactions, and ensuring system functionality. Their objective is to maintain the integrity of the system and assist users as needed. |
- Payment Gateway--An external service that processes payments for transactions initiated by customers. Its objective is to securely handle payment information and confirm transaction success or failure. |

Putting Together Requirements

- Stakeholder Interviews: Engage with stakeholders to gather insights on their needs.
- Surveys and Questionnaires: Collect data from potential users to understand their preferences.
- Workshops: Conduct collaborative sessions to refine requirements

What is a Class?

- A class is a model element that represents an object or a set of objects that share a common structure and behavior:
- **Attributes**: The characteristics of the object, such as brand, model, year, and color
- **Operations**: The things the object can do, such as accelerating, decelerating, stopping, and starting
- **Relationships**: How the object relates to other objects, such as aggregation, generalization, association, and dependency

What is a UML diagram?

- A UML diagram is a way to visualize systems and software using Unified Modeling Language (UML).
- Software engineers create UML diagrams to understand the designs, code architecture, and proposed implementation of complex software systems. UML diagrams are also used to model workflows and business processes.
- They are categorized into two main types: **structural diagrams** and **behavioral diagrams**.

Structural Diagrams

Structural diagrams: provide a static view of the system, illustrating its components and their relationships. The main types include:

- **Class Diagram**: Depicts the classes in a system, their attributes, operations, and relationships. It is fundamental for object-oriented design.
- **Object Diagram**: Shows instances of classes at a specific moment, illustrating the state of the system and relationships between objects.
- **Component Diagram**: Represents the organization and dependencies among software components, helping to visualize system architecture.

Structural Diagrams

- **Deployment Diagram**: Illustrates the physical deployment of artifacts on nodes, showing how software interacts with hardware in a distributed environment.
- **Package Diagram**: Organizes classes into packages and shows dependencies between them, facilitating better management of complex systems.
- **Composite Structure Diagram**: Displays the internal structure of a class and how its parts interact with other components.

Behavioral Diagrams

- **Behavioral diagrams** focus on the dynamic aspects of a system, detailing how components interact and behave over time. Key types include:
- **Use Case Diagram**: Represents the interactions between actors (users or systems) and use cases (functional requirements), providing an overview of system functionality.
- **Sequence Diagram**: Illustrates how objects interact in a particular scenario over time, focusing on the order of message exchanges.
- **Activity Diagram**: Visualizes workflows within a system, showing the sequence of activities and decision points.

Behavioral Diagrams

- **State Machine Diagram**: Models the states an object can be in and the transitions between those states in response to events.
- **Communication Diagram**: Similar to sequence diagrams but emphasizes the relationships between objects and messages exchanged among them.
- **Timing Diagram:** Depicts how objects interact over time, focusing on timing constraints and duration of interactions.
- **Interaction Overview Diagram**: Provides a high-level view of interactions within a system, summarizing how various components work together.